P3288R3
std::elide

New Proposal, 2024-06-27

This version:
http://virjacode.com/papers/p3288r3.htm

Latest version:
http://virjacode.com/papers/p3288.htm

Author:
Thomas PK Healy <healytpk@vir7ja7code7.com> (*Remove all sevens from email address*)

Audience:
SG17, SG18

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

Add a new class to the standard library to make possible the emplacement of an unmovable-and-uncopyable *prvalue* into types such as `std::optional`, `std::variant` and `boost::static_vector` -- without requiring an alteration to the definitions of these classes.

Table of Contents

> NOTE: A change is required to the core language.

§ 1. Introduction

While it is currently possible to return an unmovable-and-uncopyable class by value from a function:

```
std::counting_semaphore<8> FuncReturnsByValue(unsigned const a, unsigned co
{
    return std::counting_semaphore<8>(a + b);
}
```

It is not possible to emplace this return value into an `std::optional`:

```
int main()
{
    std::optional< std::counting_semaphore<8> > var;

    var.emplace( FuncReturnsByValue(1,2) );  // compiler error
}
```

This paper proposes a solution to this debacle, involving an addition to the standard library, along with a change to the core language.

§ 2. Motivation

§ 2.1. `emplace( FuncReturnsByValue() )`

There is a workaround to make this possible, and it is to use a helper class with a conversion operator:

```
int main()
{
    std::optional< std::counting_semaphore<8> > var;

    struct Helper {
        operator std::counting_semaphore<8>()
        {
            return FuncReturnsByValue(1,2);
        }
    };

    var.emplace( Helper() );
}
```

This is possible because of how the `emplace` member function is written:

```
template<typename... Params>
T &emplace(Params&&... args)
{
    . . .
    ::new(buffer) T( forward<Params>(args)... );
    . . .
}
```

or:

```
template<typename... Params>
T &emplace(Params&&... args)
{
    . . .
```

```
    std::construct_at<T>( buffer, forward<Params>(args)... );
    . . .
}
```

The compiler cannot find a constructor for `T` which accepts a sole argument of type `Helper`, and so it invokes the conversion operator, meaning we effectively have the constructor of `T` invoked as follows:

```
T( FuncReturnsByValue(1,2) );
```

In this situation, where we have a *prvalue* returned from a function, we have guaranteed elision of a copy/move operation. This proposal aims to simplify this technique by adding a new class to the standard library called `std::elide` which can be used as follows:

```
int main()
{
    std::optional< std::counting_semaphore<8> > var;

    var.emplace( std::elide(FuncReturnsByValue,1,2) );
}
```

§ 3. Possible implementation

```
#include <functional>   // invoke
#include <type_traits>  // false_type, invoke_result, is_reference, is_same
#include <utility>      // declval, forward

namespace std {
  template<typename F_ref, typename... Params_refs>
  requires is_reference_v<F_ref> && (is_reference_v<Params_refs> && ...)
  class basic_elide {
      typedef invoke_result_t< F_ref, Params_refs... > R;
      static_assert( !is_reference_v<R>, "F must return by value" );
  protected:
      inline static constexpr bool excepts = noexcept(invoke(declval<F_ref>

      static constexpr decltype(auto) GetLambda(F_ref f, Params_refs... arg
      {
          typedef remove_reference_t<F_ref> F;
```

```
        typedef remove_pointer_t<F> funcF;  // Might be pointer-to-func c
        if constexpr ( sizeof...(Params_refs) )
        {
            // Next line returns lambda by value
            return [&f,&args...](void) noexcept(excepts) -> decltype(autc
              {
                return invoke( static_cast<F_ref>(f), static_cast<Params_
              };
        }
        else if constexpr ( is_function_v<funcF> )
        {
            return static_cast<funcF*>(f);  // returns pointer-to-func by
        }
        else if constexpr ( is_trivial_v<F> && sizeof(F)<=sizeof(void*) )
        {
            return static_cast<F>(f);  // returns small functor object by
        }
        else
        {
            return static_cast<F_ref>(f);  // returns a ref to functor ob
        }
    }

    typedef decltype(GetLambda(declval<F_ref>(),declval<Params_refs>()...
    InvokableT to_be_invoked;  // might be a lambda, might be a reference

public:
    template<typename F, typename... Params>
    constexpr explicit basic_elide(F &&arg, Params&&... args) noexcept  :
      : to_be_invoked( GetLambda( forward<F>(arg), forward<Params>(args)...

    constexpr operator R(void) noexcept(excepts)
    {
        typedef conditional_t< is_lvalue_reference_v<F_ref>, InvokableT&,
        return invoke( static_cast<InvokableTref>(to_be_invoked) );
    }

    constexpr R operator()(void) noexcept(excepts)
    {
        return this->operator R();
    }

    /* -------- Delete all miranda methods -------- */
```

```
        basic_elide(     void     ) = delete;
        basic_elide(basic_elide const & ) = delete;
        basic_elide(basic_elide       &&) = delete;
        basic_elide &operator=(basic_elide const & ) = delete;
        basic_elide &operator=(basic_elide       &&) = delete;
        basic_elide const volatile *operator&(void) const volatile = delete;
        template<typename U> void operator,(U&&) = delete;
        /* ---------------------------------------- */
    };

    template<typename F, typename... Params>
    class elide : public basic_elide<F, Params...> {
    public:
        typedef true_type tag_tempfail_ctor_soleparam;
        using basic_elide<F, Params...>::basic_elide;
    };

    template<typename F, typename... Params>
    class elide_c1 : public basic_elide<F, Params...>  {
    public:
        typedef false_type tag_tempfail_ctor_soleparam;
        using basic_elide<F, Params...>::basic_elide;
    };

    // Here come the two deduction guides:
    template<typename F, typename... Params>
    elide(F&&,Params&&...) -> elide<F&&,Params&&...>;  // explicit deduction
    template<typename F, typename... Params>
    elide_c1(F&&,Params&&...) -> elide_c1<F&&,Params&&...>;  // explicit dedu

    template<typename T>
    concept has_tag_tempfail_ctor_soleparam_true = T::tag_tempfail_ctor_solep
}
```

Thoroughly tested on GodBolt: https://godbolt.org/z/o86nG1z4h
You can comment out **Line #134** in the godbolt to see the effect of the core language change.

§ 4. Design considerations

§ 4.1. template constructor

The above implementation of `std::elide` will not work in a situation where a class has a constructor which accepts a specialisation of the template class `std::elide` as its sole argument, such as the following `AwkwardClass`:

```
class AwkwardClass {
    std::mutex m;  // cannot move, cannot copy
public:
    template<typename T>
    AwkwardClass(T &&arg)
    {
        cout << "In constructor for AwkwardClass, \n"
                "type of T = " << typeid(T).name() << endl;
    }
};

AwkwardClass ReturnAwkwardClass(int const arg)
{
    return AwkwardClass(arg);
}

int main(int const argc, char **const argv)
{
    std::optional<AwkwardClass> var;
    var.emplace( std::elide(ReturnAwkwardClass, -1) );
}
```

The above program will print out:

```
--
In constructor for AwkwardClass,
type of T = std::elide< AwkwardClass, AwkwardClass (&)(int), int&& >
--
```

The problem here is that the constructor of `AwkwardClass` has been instantiated with the template parameter type `T` set to a specialisation of `std::elide`, when really we wanted `T` to be set to `int`. We want the following output:

```
--
In constructor for AwkwardClass,
type of T = int
--
```

A workaround here is to apply a constraint to the constructor of `AwkwardClass` as follows:

```
template<typename... Params>
requires (!( (1u==sizeof...(Params)) && (std::has_tag_tempfail_ctor_solepa
AwkwardClass(Params&&... arg)
{
    ( std::cout << "In constructor for AwkwardClass, type of T = " << type
}
```

In order that class definitions do not have to be altered in order to apply this constraint to template constructors, this proposal makes a change to the core language to prevent the constructor of any class from having a specialisation of `std::elide` as its sole parameter. This constraint is achieved by detecting the presence of a `typedef` called `tag_tempfail_ctor_soleparam` for which `tag_tempfail_ctor_soleparam::value` evaluates to `true`.

The programmer can write their own custom class to use instead of `std::elide`, and in order to take advantage of the core language feature which prevents the instantiation of constructors, the programmer can give their custom class an accessible `typedef` as follows:

```
typedef std::true_type tag_tempfail_ctor_soleparam;
```

Furthermore, if the programmer wishes to use `std::elide` in a scenario where a template constructor is desired to be instantiated with an elider as its sole parameter type, the alternative class `std::elide_c1` is provided for this purpose.

§ 5. Alternatives

§ 5.1. emplace_invoke

An alternative would be to add a method called `emplace_invoke` to classes like `std::optional` and `std::variant`, as follows:

```
template<typename F, typename... Params>
T &emplace_invoke(F &&f, Params&&... args)
{
    . . .
    ::new(buffer) T( std::forward<F>(f)( std::forward<Params>(args)... ) );
    . . .
}
```

The benefit of `emplace_invoke` is that it doesn't require a change to the core language. The benefit of `std::elide` is that pre-existing header files which implement or use the `emplace` method can be used with `std::elide` without requiring an alteration to the header file. (For example, there would be no need to alter the *Boost* header file for `boost::static_vector` in order to use it with `std::elide`).

§ 6. Proposed wording

The proposed wording is relative to [N4950].

In subclause 13.10.3.1.11 [temp.deduct.general], append a paragraph under the heading *"Type deduction can fail for the following reasons:"*

```
11 -- Attempting to instantiate a constructor that has exactly one parameter
      and the sole parameter is a class type which contains an accessible
      typedef called 'tag_tempfail_ctor_soleparam' whose 'value' evaluates
      to true, for example:
                typedef std::true_type tag_tempfail_ctor_soleparam;
```

§ 7. Impact on the standard

This proposal is a library extension combined with a change to the core language. The change to the core language is a paragraph to be added to 13.10.3.1.11 [temp.deduct.general]. The addition has no effect on any other part of the standard.

§ 8. Impact on existing code

No existing code becomes ill-formed. The behaviour of all existing code is unaffected.

§ 9. Revision history

R2 => R3

- Possible implementation changed to store a lambda instead of using a tuple full of references, with optimisations for function pointers and very small trivial functors.
- Two separate classes `std::elide` and `std::elide_c1` so that the programmer can use the latter in a scenario where a template constructor is desired to be instantiated with an elider as its sole parameter type.
- Chaining of eliders is now possible with `operator()`.
- `tag_elide` renamed to `tag_tempfail_ctor_soleparam` to make it generic so that it can be used for other purposes in the future.
- The example of deriving a class from `std::elide` is removed as it's unnecessary because we have `std::elide_c1`.

R1 => R2

- New section entitled '*Alternatives*' to suggest that `std::optional` and `std::variant` could be given an `emplace_invoke` method.
- `std::elide` can be used in a constant-evaluated context.
- `std::construct_at` is given as an alternative to *placement* `new` (note that the former can be used in a constant-evaluated context).
- The example of `std::any` is removed because `T` must be copy-constructible.
- The example of `boost::static_vector` is added.
- The proposed wording for the standard clarifies that the accessible typedef is called `tag_elide`

- Correction of typo, `is_elider` -> `has_tag_elide_true`
- Correction of typo, `typename T` -> `typename... Params`

R0 => R1

- The class `std::elide` is not defined as `final`
- Template instantiation only fails if the constructor has exactly one parameter.
- Template instantiation fails based upon the presence of a `typedef` tag whose `value` evaluates to `true`.

§ 10. Acknowledgements

For their feedback and contributions at the BSI (*British Standards Institution*) C++ committee meetings:

Gašper Ažman, Oliver Rosten, Lénárd Szolnoki, Matthew Taylor

For their feedback and contributions on the mailing list **std-proposals@lists.isocpp.org**:

Jens Mauer, Jason McKesson, Ville Voutilainen, Jonathan Wakely

And for their insightful blogs:

Andrzej Krzemieński, Arthur O'Dwyer

§ References

§ Normative References

[N4950]
Thomas Köppe. *Working Draft, Standard for Programming Language C++*. 10 May 2023. URL: https://wg21.link/n4950